

1. Idempotência

Por que utilizar (**transaction_id + event**) e não apenas **transaction_id**?

Eu utilizaria a chave composta (**transaction_id, event**) porque um mesmo pedido pode gerar diversos eventos diferentes ao longo do tempo, mantendo o mesmo **transaction_id**.

Exemplo:

- order.created -> transaction_id = TX123
- order.approved -> transaction_id = TX123
- order.refunded -> transaction_id = TX123
- order.cancelled -> transaction_id = TX123

Se a chave fosse apenas **transaction_id**, o primeiro evento recebido poderia impedir o processamento dos próximos, mesmo sendo eventos válidos e importantes para o negócio.

A inclusão do campo **event** permite registrar corretamente toda a evolução do pedido, evitando apenas o reprocessamento do mesmo evento.

Trade-off

- **transaction_id**: implementação mais simples, porém pode descartar eventos legítimos.
- **transaction_id + event**: representa melhor o ciclo de vida do pedido, com um pequeno aumento na complexidade.

Cenário de falha

Utilizando apenas **transaction_id**, um pedido aprovado e posteriormente reembolsado poderia ter o evento de refund ignorado por ser considerado duplicado.

Por outro lado, utilizando **transaction_id + event**, seria necessário garantir uma padronização dos nomes dos eventos para evitar que duas nomenclaturas diferentes para a mesma ação sejam tratadas como eventos distintos.

2. Criptografia

Este foi um dos tópicos que exigiu pesquisa durante o desenvolvimento do desafio. Como foi meu primeiro contato prático com criptografia aplicada a webhooks, busquei entender as diferenças entre AES-256-CBC e AES-256-GCM, os cenários em que cada um é mais utilizado e os riscos de segurança associados. Esse estudo foi importante para

compreender não apenas a implementação do decrypt do gateway Grummer, mas também os critérios para escolher uma solução adequada para novas integrações.

AES-256-CBC vs AES-256-GCM

Para um novo webhook da GEX, eu escolheria AES-256-GCM.

O AES-256-CBC protege o conteúdo da mensagem, mas não garante sua integridade por padrão. Em implementações incorretas pode ser mais vulnerável a ataques relacionados à manipulação do ciphertext e problemas de padding.

O AES-256-GCM fornece criptografia autenticada, protegendo simultaneamente a confidencialidade e a integridade dos dados. Além de descriptografar, ele também valida se o conteúdo foi alterado durante o transporte.

Trade-off

- AES-256-CBC útil para manter compatibilidade com integrações legadas.
- AES-256-GCM mais seguro e mais simples para novos contratos.

Por esse motivo, manteria CBC apenas para integrações já existentes e utilizaria GCM para novas implementações.

3. Backpressure

Como proteger o sistema caso o provedor SMS passe a falhar com 90% de erro?

A primeira medida seria garantir o isolamento do problema.

A falha do SMS não deve impactar:

- API de recebimento
- Consumer principal
- Banco de dados
- Outros canais de distribuição

Para isso, manteria filas independentes para cada canal, permitindo que apenas o fluxo SMS seja afetado.

Também utilizaria:

- Retry exponencial
- Limite de tentativas
- Dead Letter Queue (DLQ)
- Controle de concorrência
- Alertas operacionais

Vale ressaltar que a arquitetura desenvolvida no teste técnico já segue esse princípio de isolamento e desacoplamento. Cada etapa da esteira possui sua própria responsabilidade e comunicação assíncrona através do RabbitMQ, permitindo que problemas em um canal específico, como o SMS, não interrompam o recebimento de webhooks, a persistência dos dados ou o processamento dos demais fluxos. Essa separação reduz o impacto de falhas localizadas e aumenta a resiliência geral da aplicação.

Por que RabbitMQ + retry exponencial não basta?

Se o provedor permanecer indisponível por um longo período, o sistema pode ficar ocupado reprocessando mensagens com baixa chance de sucesso, aumentando backlog e consumo de recursos.

Por isso, além dos retries, é importante possuir DLQ, monitoramento e mecanismos para impedir que um único canal degrade toda a esteira.

Trade-off

- Reprocessar ajuda em falhas temporárias.
- Reprocessar excessivamente pode agravar o problema quando a falha é persistente.

Durante o desenvolvimento do teste técnico foi possível visualizar esse comportamento na prática ao simular falhas no distribuidor SMS. Embora o retry exponencial ajude a absorver indisponibilidades temporárias do provedor, ele não resolve cenários onde a falha é persistente. Nesses casos, as mensagens continuam sendo reprocessadas sem sucesso, aumentando o backlog e consumindo recursos dos workers. Por esse motivo, além dos retries, implementei DLQ para isolamento das mensagens problemáticas e métricas via Prometheus para monitoramento da esteira, permitindo identificar rapidamente quando um canal começa a degradar e evitando que o problema se propague para o restante do sistema.

4. Migração entre Linguagens

Quando valeria migrar Receiver + Decrypt de Python para Go?

Eu consideraria essa migração caso existissem evidências claras de que essa etapa se tornou um gargalo operacional.

Sinais que indicariam migração

1. Alto consumo de CPU no decrypt

Se o volume de payloads criptografados crescer significativamente e a descryptografia passar a consumir grande parte dos recursos da aplicação. Nesse cenário, o Go poderia ser considerado como opção, por oferecer excelente suporte à concorrência através de **Goroutines**, que são extremamente leves, além de apresentar baixo consumo de memória e boa eficiência no processamento de operações intensivas de CPU.

2. Latência elevada no recebimento

Mesmo com filas e processamento assíncrono, o receiver passa a apresentar tempos de resposta acima do esperado.

3. Necessidade de maior concorrência com menor custo

Se a GEX precisar processar volumes muito maiores de webhooks mantendo baixo consumo de memória e infraestrutura.

Sinais que NÃO indicariam migração

1. O gargalo está em outro lugar

Se o problema estiver em consultas SQL, RabbitMQ ou integrações externas, trocar de linguagem não resolverá a causa raiz.

2. O volume atual é bem atendido

Se a aplicação já responde rapidamente e o processamento principal está desacoplado através das filas, a migração pode não trazer ganhos relevantes.

3. O time possui forte domínio em Python

A produtividade, manutenção e onboarding da equipe também devem ser considerados na decisão.

Trade-off

Migrar para uma linguagem mais performática pode trazer ganhos de eficiência, porém aumenta a complexidade operacional, o custo de manutenção e a necessidade de especialização da equipe.

Por esse motivo, minha decisão seria sempre baseada em métricas reais de produção e não apenas em preferência tecnológica (caso houvesse).